

CS 241 Data Organization using C Parsing Command Line Arguments, Error Checking, and Conversion between Binary, Decimal, and Hexadecimal

Joseph Haugh

Spring 2024

1 Reading Command-Line Arguments in a C Program

Many C programs created and used in both industry and in academia are designed with a command-line human interface. Kernighan and Ritchie provide a description of how to program this in section 5.10: Command-line Arguments.

2 Requirements

Write a C program, `convert.c`, with the usage:

usage:

`convert IN OUT SIZE NUMBER`

IN:

- inB NUMBER is binary.
- inD NUMBER is decimal.
- inH NUMBER is hexadecimal.

OUT:

- outB Output will be in binary.
- outD Output will be in decimal.
- outH Output will be in hexadecimal.

SIZE:

- 8 input is an unsigned 8-bit integer.
- 16 input is an unsigned 16-bit integer.
- 32 input is an unsigned 32-bit integer.
- 64 input is an unsigned 64-bit integer.

NUMBER:
number to be converted.

2.1 Errors

When bad input is given, your program must print the appropriate error message followed by the usage message shown above. (See the example output for the possible error messages.)

2.2 Parsing Strings as Numbers

You might find it helpful to use the `strtoul` function of `<stdlib.h>`. This function can convert a string of digits into an unsigned long value.

2.3 Output Format for Binary Numbers

Every 4 binary numerals must be separated by a space. All leading zeros must be displayed. Thus, an 8 bit integer must display 8 bits and a 16 bit integer must display 16 bits.

Command	Output
<code>./convert -inD -outB -8 63</code>	0011 1111
<code>./convert -inD -outB -16 63</code>	0000 0000 0011 1111

If a number is too large to be represented in the specified number of bits, then output only the lower bits of that number. For example, the number 259 is $2^8 + 2^1 + 2^0$; however, output as an 8-bit integer drops the 2^8 place:

Command	Output
<code>./convert -inD -outB -8 259</code>	0000 0011
<code>./convert -inD -outB -16 259</code>	0000 0001 0000 0011

The modulus operator is useful in deciding when to leave a space.

2.4 Output Format for Decimal Numbers

Decimal numbers must be formatted with comma thousand separators and with leading spaces (not leading zeros). For example, the largest unsigned 8-bit number is 11111111 (base 2) = 255 (base 10). Since 255 has three digits, then all 8-bit numbers should be printed right justified within a field of 3 characters:

Command	Output
<code>./convert -inB -outD -8 11111111</code>	255
<code>./convert -inB -outD -8 1111</code>	15
<code>./convert -inB -outD -8 11</code>	3

Similarly, the largest unsigned 16 bit number (1111111111111111_2) is 65,535 in base 10. This contains 5 digits plus a comma. Thus, all 16-bit decimal numbers must be right justified in a field of 6 characters:

Command	Output
<code>./convert -inB -outD -16 1111111111111111</code>	65,535
<code>./convert -inB -outD -16 11111111</code>	255
<code>./convert -inB -outD -16 1111</code>	15
<code>./convert -inB -outD -16 11</code>	3

WARNING: ANSI C does not include a format option for automatically adding comma thousand separators nor does the gcc used on `moons.unm.edu`. This format option is standard in C++ and in many expanded versions of C. If you are writing your code using a different C compiler and turn in code with such an option specified, your program will most likely not compile when tested and you will get a zero for the assignment.

It is not that hard to write your own code for outputting numbers with comma thousand separators. However, this is such a common task that a quick Google search should return a solution in ANSI C. It is ok for you to use such code. *If you do use any code you did not write, be sure to avoid plagiarism by giving proper credit for the source.*

2.5 Output Format for Hexadecimal Numbers

Every byte must be separated by a space. All leading zeros must be displayed. Thus, an 8 bit integer must display 1 byte and a 16 bit integer must display 2 bytes.

Command	Output
<code>./convert -inD -outH -8 63</code>	3f
<code>./convert -inD -outH -16 63</code>	00 3f

If a number is too large to be represented in the specified number of bits, then output only the lower bits of that number. For example, the number 259 is $2^8 + 2^1 + 2^0$; however, output as an 8-bit integer drops the 2^8 place:

Command	Output
<code>./convert -inD -outH -8 259</code>	03
<code>./convert -inD -outH -16 259</code>	01 03

Note: I gave you an example of formatting a hexadecimal number with spaces. You may use that as a starting point when formatting your output.

3 Grading Rubric (total of 35 points)

-2 point : The program does not start with a comment stating the students first and last name and/or the source file is not named correctly.

-2 points : Program compiles with warnings on moons.cs.unm.edu using `/usr/bin/gcc` with the `-Wall -ansi -pedantic` options (and options for any standard libraries you may want to use such as `-lm` for `math.h`). If you do use a standard library that requires a linking option, please include this as a note when you submit in Canvas.

5 points : All code follows the CS-241 standards (including comments).

30 points : Passes 60 known test cases: 0.5 point each. The test file is `testconvert.sh` and the output file is `testconvert.out`. Note, unlike past test files, this test file is not a data file to be input into your program by redirecting the standard input stream. The file `testconvert.sh` is a Linux shell script file (it is a text file containing a sequence of Linux commands). Note: for this shell script to run, it must have execution permission turned on:

```
chmod u+x testconvert.sh
```

Also, the script expects your executable to be named `convert` (not `a.out`), so make sure you specify that when you compile.

```
gcc -Wall -ansi -pedantic -o convert convert.c
```

I've provided you with a simple makefile that uses these options so you won't have to keep typing them.